MD5-Kollisionen in PostScript

Grundlagenpraktikum zur IT-Sicherheit
(WiSe 2023/2024)

Lehrstuhl für Netz- und Datensicherheit
Ruhr-Universität Bochum

Nico S**** Sabri Öztürk

1. Mai 2026

Zusammenfassung

In diesem Versuch soll ein Einblick in MD5-Kollisionen gewonnen werden. Durch das Ausnutzen dieser Kollisionen können zwei verschieden aussehende PostScript Dateien den gleichen MD5 Hash haben. Was daraus resultieren kann, wir im Folgenden erläutert.

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Glossar 1

2 Aufbau **2**

3 Durchführung **3**

3.1 Anzeigen der Datei 3

3.2 Erstellung der zweiten, versteckten Überweisung 3

3.3 Einbau des Schalters zum Auswechseln der Überweisungen 4

3.4 Zerteilen der PostScript-Datei 5

3.5 Kompilierung des IV-Finders 6

3.6 Finden des IV 8

3.7 Finden der Kollision 8

3.8 Zusammenbau 9

4 Responsible Discolsure **11**

5 Zusammenfassung und Fazit **12**

6 LLM **13**

1 Einleitung

In der heutigen Zeit werden Dokumente aller Art meist digital übertragen. Das ist hilfreich, da es deutlich schneller von statten gehen kann, als beispielsweise einen Brief zu schreiben. Doch stellt sich dann die Frage, wie ein Dokument als authentisch gilt und einer Person zugeordnet werden kann.

Dokumente die in Papierformat vorliegen können mit einer Unterschrift, einem Stempel oder einem Siegel versehen werden. Bei digitalen Dokumenten ist dies jedoch nicht so leicht. Eine Datei kann Bit für Bit kopiert werden, sodass eine Unterschrift die auf einem PDF-Dokument zu sehen ist gleich mit kopiert wird. Sie verliert so ihre Gültigkeit.

Hier sind digitale Signaturen nützlich. Diese sind so ausgelegt, dass sie beliebige Daten in Binärformat einer bestimmten Person zugeordnet werden können. Da Dokumente teilweise sehr groß werden können und das Signieren von großen Dateien unpraktisch sein kann, können auch die Hashwerte von Dokumenten signiert werden. Dies bietet Vor- aber auch Nachteile.

Als Hashwerte werden die Ausgaben von Hashfunktionen bezeichnet. Diese Hashfunktionen bilden eine Eingabe beliebiger Länge auf eine Ausgabe fester Länge ab. Zudem sorgt schon eine kleine Änderung in der Eingabe für eine komplett andere Ausgabe. Wenn nun der Hashwert eines Dokuments signiert wird, muss nur eine feste, und in der Regel kleine, Anzahl an Bytes signiert werden.

Diese Anwendung birgt jedoch auch Gefahren. Da es mehr Möglichkeiten für Eingaben als Ausgaben gibt, müssen zwangsweise unterschiedliche Eingaben auf gleiche Ausgaben abgebildet werden. So können unterschiedliche Dokumente den gleichen Hashwert haben und so wäre auch die gleiche Signatur für beide Dokumente gültig. Diese Eigenschaft soll im folgenden Versuch ausgenutzt werden, um eine gestellt Überweisung zu fälschen.

1.1 Glossar

Hashfunktion Funktion die eine Eingabe beliebiger Länge auf eine Ausgabe fester Länge abbildet

Hashwert Name für Ausgabe einer Hashfunktion

Hashkollision Zwei unterschiedliche Eingaben haben den gleichen Hashwert

2 Aufbau

Die folgenden Versuche wurden im Netzlabor ID 2/168 unternommen.
Wir führen den Versuch zu zweit als Gruppe 43 am PC mit der IP-Adresse 192.168.1.42 durch.
Das Betriebsystem ist ein Ubuntu 22.04 LTS.

Es wurden verschiedene Programme und Befehle genutz. Diese sind:

Software	**Version**
open (xdg-open)	1.1.3
ghex	3.41.1
GNU Nano	6.2
md5sum	(GNU coreutils) 8.32
GNU Make	4.3
Dokumentenbetrachter	42.3

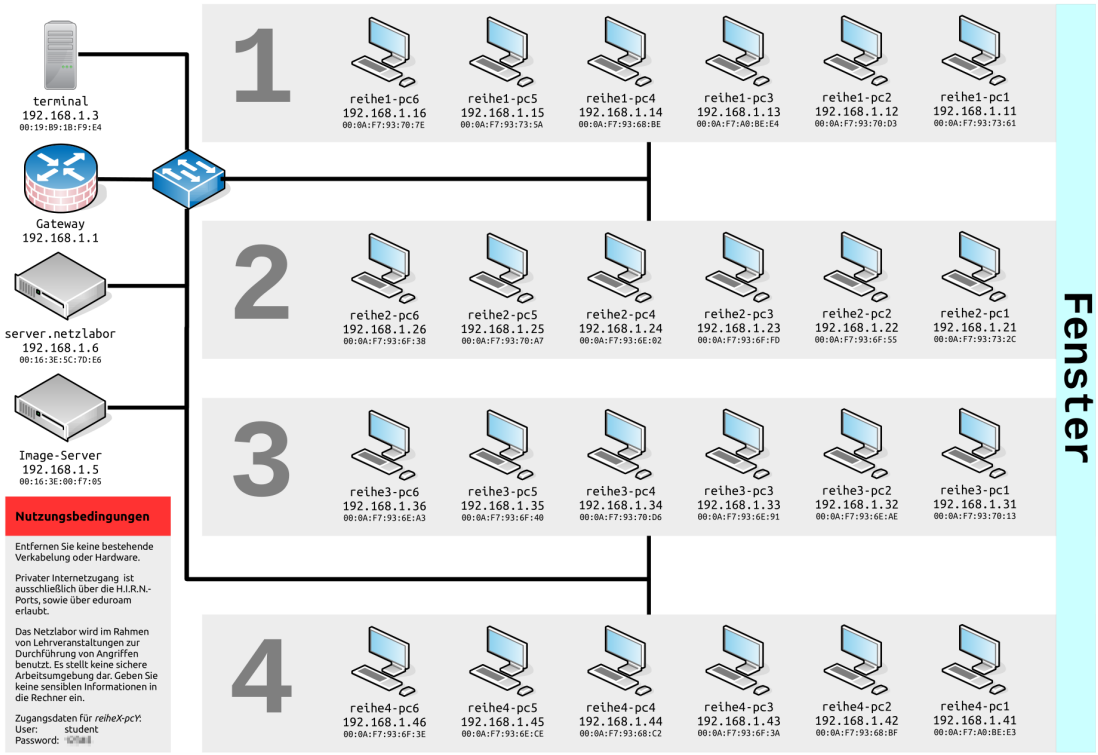

Abbildung 1: Netzplan Raum ID 2/168, Stand 01.05.2017

Die Aufgabenstellung [2] wurde bereutgestellt übe die Veranstaltungsseite [1]

3 Durchführung

Im Folgenden werden die einzelnen Aufgaben durchgeführt, erläutert und das Ergebnis präsentiert.

3.1 Anzeigen der Datei

Für diesen Versuch liegt eine gestellte Überweisung als PostSCript Datei vor. PostScript ist eine turingvollständige Setienentwicklungssprache. Mit dieser Sprache kann das Aussehen und der Inhalt von Dokumenten festgelegt werden. Die Überweisungsdatei hat den Namen *ueberweisung.ps* und den in Abbildung 2 zusehenden Inhalt.

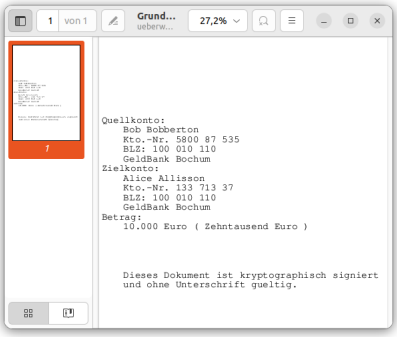

Abbildung 2: Inhalt der Datei 'ueberweisung.ps'

3.2 Erstellung der zweiten, versteckten Überweisung

In PostScript können Inhalte existieren, die nicht angezeigt werden. Um die Überweisung für den später erfolgenden Angriff zu präparieren wird zusätzlich zur ersten Definition, *dokument1*, welche die in Abbildung 2 sichtbare Überweisung enthält, eine weitere Definition *dokument2* erstellt. Diese Enthält eine Kopie der ersten Definition, jedoch werden Informationen über das Zielkonto geändert. Damit die Zahlung auf ein anderes Konto überwiesen wird, wird der Name des Zielkontos auf 'Oskar' und die Kontonummer auf '353 735 37' geändert. Zudem wird der Aufruf 'dokument1' auf 'dokument2' geändert, damit der neue Inhalt angezeigt wird (siehe Abbildung 3).

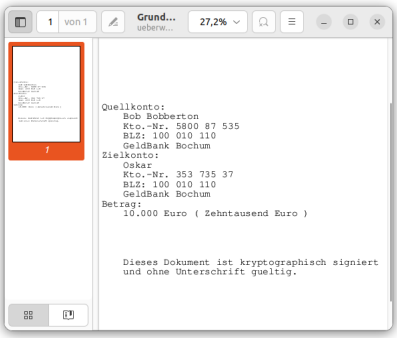

Abbildung 3: Inhalt der Datei 'ueberweisung.ps' nach Änderungen

3.3 Einbau des Schalters zum Auswechseln der Überweisungen

In PostScript existieren IF-Verzweigungen. Um den Inhalt des Dokuments einfach verändern zu können, kann eine solche IF-Verzweigung genutzt werden. Die folgende Verzweigung wird an der Stelle in das Dokument eingebaut, wo der Aufruf 'dokument2' steht.

```
(V1)
(V2)
eq
{ dokument2 }
{ dokument1 }
ifelse
```

Hier werden die zwei Konstanten *V1* und *V2* mit einander verglichen. Die Zeile *eq* sagt aus, dass die beiden Konstanten gleich (englisch equal) sein müssen. Wenn dies der Fall ist, wird der Aufruf in den ersten Klammern ausgeführt, wenn nicht, der Aufruf in den letzteren Klammern. So kann zwischen den verschiedenen Inhalten gewechselt werden.

Wird die Datei mit den Änderungen gespeichert und im Dokumentenbetrachter geöffnet, so kann die originale Überweisung wie in Abbildung 2 gesehen werden.

Wenn nun die Konstante *V1* auf *V2* geändert und gespeichert wird, kann nach dem öffnen der Datei im Dokumentenbetrachter der Inhalt wie in Abbildung 3 gesehen werden. Die geänderte Datei enthält nun also die Zeilen:

```
(V2)
(V2)
eq
{ dokument2 }
{ dokument1 }
ifelse
```

3.4 Zerteilen der PostScript-Datei

Um später den Angriff durchführen zu können, muss die Überweisung *ueberweisung.ps* in passende Teile aufgeteilt werden. Die Konstanten *V1* und *V2* sollen später durch die Kollisionsblöcke ausgetauscht werden. Da dies unter Umständen mehrere Versuche braucht, ist ein Programm gegeben, welches diese Aufgabe übernimmt. Das Programm erwartet, dass in dem Unterordner *components/* gewisse Dateien vorhanden sind. Die PostScript Datei wird aufgeteilt in die Dateien *start*, *center* und *end*. Start enthält alles, was vor der Konstante *V1* in der PostScript Datei steht. Die Datei muss ein Vielfaches von 512 Bit lang sein. Dies ist notwendig, da die MD5-Blockgröße ausgefüllt werden muss. Ein Block in MD5 ist 512 Bit groß. Der CBC Modus nimmt einen Initialisierungsvektor *IV*, verrechnet diesen mit dem eingegebenen Block und nimmt die Ausgabe dieser Rechnung als neuen *IV* für den nächsten Block. Wenn nun die Datei *start* ein vielfaches von 512 Bit lang ist, wird der Hashwert dieser Datei der gleiche bleiben, auch, wenn danach Inhalte eingefügt werden. Das ist wichtig für die erfolgreiche Durchführung des Angriffs. Um das zu erreichen, werden vor dem letzten Zeichen der Datei, also '(', Leerzeichen eingefügt. Diese haben den Hexwert *0x20*. Die Datei *start* kann in Abbildung 4 betrachtet werden. Zudem ist dort sichtbar, dass vor dem letzten Zeichen '(', *0x28* in Hex, 51 Leerzeichen *0x20* eingefügt wurden, um die korrekte Länge der Datei zu gewährleisten.

Die Datei *center* soll die Zeichen zwischen *V1* und *V2*, also die Zeichenfolge *)\n(*, wobei *\n* ein Zeilenumbruch ist, enthalten. Ein Zeilenumbruch in hexadezimaler Schreibweise ist *0x0A*. In Abbildung 5 kann der Inhalt der Datei gesehen werden.

Zuletzt soll die Datei *end* alles enthalten, was nach ')' nach *V2* kommt. Dies ist der Hauptteil der IF-Verzweigung, sowie das anzeigen der Datei. Der Inhalt der Datei kann in Abbildung 6 gesehen werden.

Das Programm zum Zusammenfügen der Dateien erwartet noch die beiden Dateien *collision_block1* und *collision_block2*. Diese werden jedoch von einem anderen Programm automatisch erstellt und müssen nicht per Hand bearbeitet werden.

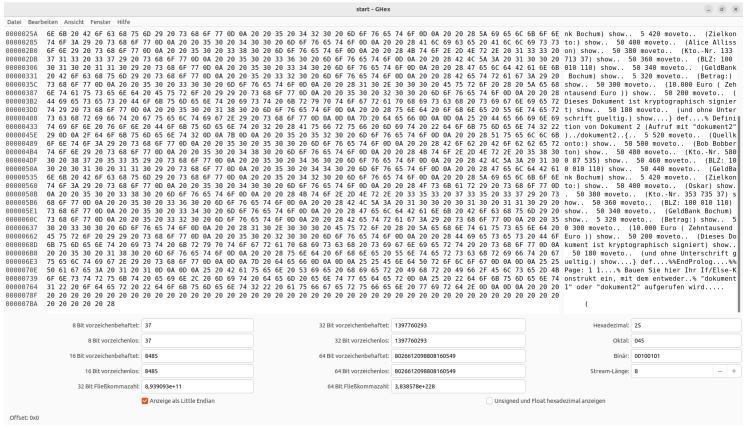

Abbildung 4: Hex Ansicht der Datei 'start'

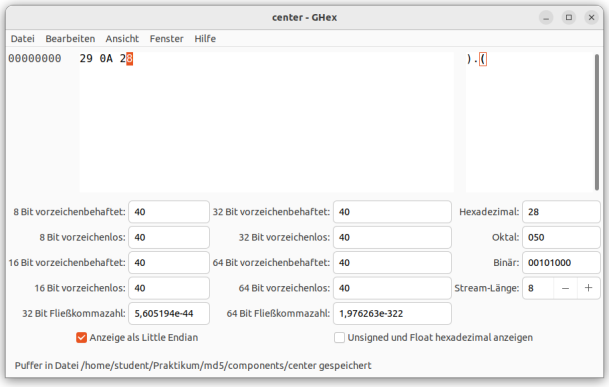

Abbildung 5: Hex Ansicht der Datei 'center'

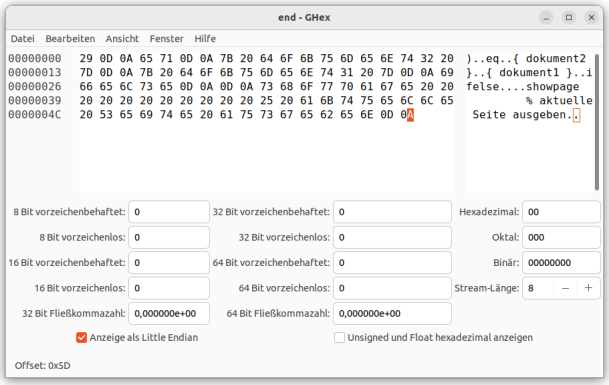

Abbildung 6: Hex Ansicht der Datei 'end'

3.5 Kompilierung des IV-Finders

Damit, wie in der vorherigen Aufgabe beschrieben, die Datei *start* als Initialisirungsvektor für die restliche Berechnung des Hashwertes gilt, muss die Hashfunktion *md5* verändert werden, da *md5* in der standard Version immer ein Padding und die Länge der Nachricht anhängt. Da der berechnete Wert von *start* jedoch als neue Eingabe verwendet werden soll, darf die zusatz Information nicht angehängt werden. Um die Funktionalität zu entfernen, haben wir die Datei *md5.c* gegeben. Die Padding-Funktionalität (siehe Abbildung 7) kann durch auskommentieren entfernt werden (siehe Abbildung 8). Zudem muss das anhängen der Länge verhindert werden. Auch dies kann durch auskommentieren der entsprechenden Stellen erreicht werden (siehe Abbildung 9). Um die Datei ausführen zu können um den IV zu berechnen, muss die Datei kompiliert werden. Dies kann mit Hilfe der vorhandenen Datei *MakeFile* erreicht werden. Um die Datei auszuführen wird eine Konsole im aktuellen Verzeichnis geöffnet. Nun kann der Befehl *make* genutzt werden, um die in der Datei *MakeFile* vorhandenen Befehle automatisch auszuführen.

```
student@Reihe4-PC2:~/Praktikum/md5$ make
gcc -o md5 -m32 md5.c
gcc -o coll_finder -m32 md5_vk1_single.c -w
```

Da es keine Fehlermeldungen gibt, wurde die Datei erfolgreich kompiliert und kann nun als *md5* gefunden und mit *./md5* ausgeführt werden.

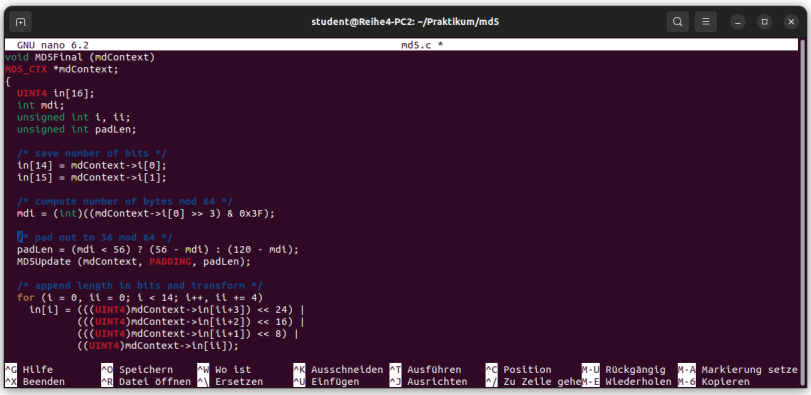

Abbildung 7: Ausschnitt der Datei 'md5.c'

Abbildung 8: Ausschnitt der Datei 'md5.c'

Abbildung 9: Ausschnitt der Datei 'md5.c'

3.6 Finden des IV

Mit dem nun modifizierten Programm können wir den IV bestimmen, der zu der Datei *start* gehört und später verwendet werden muss. Das Programm kann mit dem Befehl *./md5* ausgeführt werden. Als Parameter kann die Datei übergeben werden, die als Eingabe der Hashfunktion dient. Das Programm gibt den hashwert als einen Byte-String im *Little-Endian* Format. Es steht also in der Ausgabe das niederwertigste Byte ganz Links. Das Programm, welches für uns eine Kollisionfindet, erwartet den IV jedoch in einem anderen Format. Zur Umwandlung kann das gegebene Programm *has_to_IV.sh* genutzt werden. Dieses Programm nimmt einen Hashwert als Parameter entgegen und hat vier, vier Byte große, Hexadezimalzahlen als Ausgabe. Diese vier Hexadezimalzahlen sind später die Eingaben des Programms zum finden einer Hashkollision.

```
student@Reihe4-PC2:~/Praktikum/md5$ ./md5 components/start
e62d51719b0e987f70ab9025dee545b9 components/start
student@Reihe4-PC2:~/Praktikum/md5$ ./hash_to_IV.sh e62d51719b0e987f70ab9025dee545b9
0x71512de6 0x7f980e9b 0x2590ab70 0xb945e5de
```

3.7 Finden der Kollision

Mit dem Programm *coll_finder* und den vier Hexadezimalzahlen als Eingabeparameter, kann eine Hashkollision gefunden werden. Das eine Kollision gefunden werden kann zeigt, dass die Hashfunktion MD5 nicht mehr sicher genug für heutige Standards ist. Für kryptographisch sichere Hashfunktionen muss die *starke Kollisionsresistenz* gelten. Diese besagt, dass es praktisch nicht möglich ist, zwei Nachrichten M_1 und M_2 zu finden, sodass $h(M_1) = h(M_2)$ gilt[3]. Doch genau so etwas findet das Programm. Zudem erstellt das Programm zwei Binärdateien, welche die kollidierenden Nachrichten enthalten.

```
student@Reihe4-PC2:~/Praktikum/md5$ ./coll_finder 0x71512de6 0x7f980e9b 0x2590ab70 0
    xb945e5de
The program creates text file collision_md5_HEXnumber.TXT, containing collision.
It also creates two collision1/2_md5_HEXnum.BIN, containing it as binary.
The program takes pseudorandom numbers and its behaviour is probabilistic.
If you want to have collisions in several seconds,
wait a moment and eventually stop the program and start it again.
You can restart the program from the same point using HEXnumber as a parameter.
You can give 4 HEXnums as params to specify custom init vector for MD5.
You can give 5 HEXnums as randomseed and custom init vector.

Init vector : 0x71512DE6,0x7F980E9B,0x2590AB70,0xB945E5DE

Start from X=7B8D0977 ...
Generating block 1 ...

 The first block collision took  : 44.204863 sec
Block 1 to disk: 0
Block 2 to disk: 0

 The second block collision took  : 1.564336 sec
 The first and the second blocks together took : 45.769199 sec
Generation completed.
```

Die kollidierenden Nachrichten müssen in das Verzeichnis *components* als *collision_block1* und *collision_block2* bewegt werden, damit diese mit den anderen Dateien in *components* zusammengefügt werden können.

```
student@Reihe4-PC2:~/Praktikum/md5$ ls
coll_finder                     collision_md5_7B8D0977.TXT  md5
    ueberweisung.ps
collision1_md5_7B8D0977.BIN     components            md5.c                       zeroes
collision2_md5_7B8D0977.BIN     hash_to_IV.sh         md5_vk1_single.c
collision_test.sh               Makefile              mono_crash.mem.4431.1.blob
student@Reihe4-PC2:~/Praktikum/md5$ mv collision1_md5_7B8D0977.BIN components/
    collision_block1
student@Reihe4-PC2:~/Praktikum/md5$ mv collision2_md5_7B8D0977.BIN components/
    collision_block2
```

3.8 Zusammenbau

Im letzten Schritt beobachten wir mit dem Programm *./collision_test.sh*, dass die Kollisionsprüfung für die Dateien 'collision_block1' und 'collision_block2' erfolgreich abgeschlossen wurde. Das erkennen wir an den letzten beiden Zeilen, in denen zuerst erwähnt wird, dass eine Kollision gefunden wurde, und dann die Hashwerte explizit miteinander verglichen werden.

```
student@Reihe4-PC2:~/Praktikum/md5$ ./collision_test.sh
[ ] Werde nun 64 Null-Byte hashen.
[+] Das selbst kompilierte MD5-Programm funktioniert!
[+] Die Datei 'start' ist 1984 = 31 * 64 Byte lang.
[+] Das letzte Zeichen in 'start' ist '('. Korrekt.
[+] Das erste Zeichen in 'end' ist ')'. Korrekt.
[+] 'center' beginnt mit ')' und endet mit '('. Korrekt
[+] Die Datei 'collision_block1' ist 128 Byte lang.
[+] Die Datei 'collision_block2' ist 128 Byte lang.
[+] Die beiden Kollisionsblöcke unterscheiden sich.
[ ] Werde nun die einzelnen Teile zusammenfügen.
[ ] Berechne nun die Hashes der beiden neuen, zusammengesetzten Dateien.
[+] Kollision gefunden! :D
[+] 64cdf96b8874c7639f898cb9c27a9fc4 == 64cdf96b8874c7639f898cb9c27a9fc4
```

Die beiden Zeilen im Skript "collision_test.sh", die für die Erstellung der beiden Überweisungsdateien verantwortlich sind, sind wie folgt aufgeführt. Sie verwenden den *cat*-Befehl, um die zuvor erstellten Dateien in der Reihenfolge von *start*, *collision_block1*, *center*, *collision_block1* und *end* zusammenzuführen. In der zweiten Datei wird lediglich die erste Instanz von *collision_block1* durch *collision_block2* ersetzt.

```
cat start collision_block1 center collision_block1 end > Version11.ps
cat start collision_block2 center collision_block1 end > Version21.ps
```

Zur Überprüfung der Ergebnisse öffnen wir die Dateien 'Version11.ps' und 'Version21.ps', die sich weiterhin im Ordner 'components' befinden. Wir sehen, dass sie verschiedene Ausgaben erzeugen, obwohl sie den gleichen Hashwert haben. Somit ist der Kollisionfindung erfolgreich abgeschlossen worden.

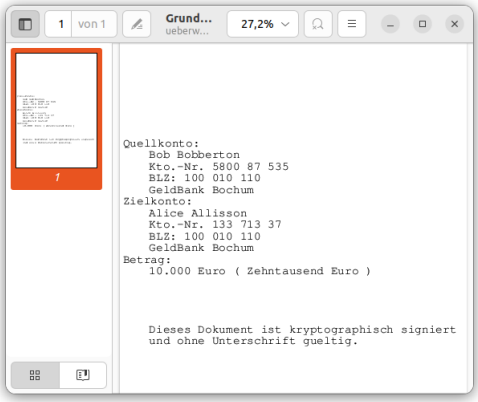

Abbildung 10: Version21.ps: Überweisung an Alice

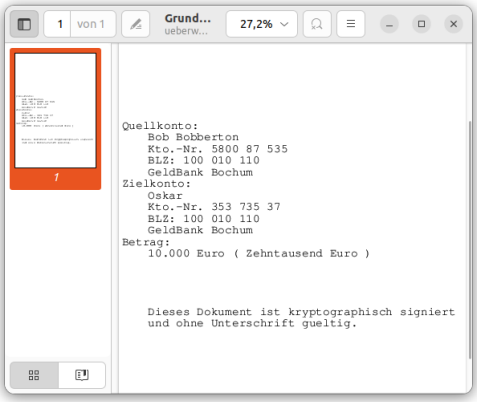

Abbildung 11: Version11.ps: Überweisung an Oskar

4 Responsible Discolsure

Sehr geehrte Damen und Herren,

bei einer kürzlichen Überprüfung Ihres Überweisungssystems sind uns einige Probleme aufgefallen. Das von Ihnen genutzte Überweisungssystem nutzt MD5-Hashfunktion um Überweisungen zu erstellen. Das nutzen von MD5 birgt Risiken.

MD5 ist anfällig gegen Kollisionsangriffe, wodurch es sich zwei verschiedene Überweisungen mit dem gleichen Hashwert erstellen lässt. Dies kann dazu führen, dass eine Datei dem Kunden den den richtigen Betrag und Begünstigten der Überweisung anzeigt, bei Ihnen jedoch eine veränderte, jedoch immer noch gültige Datei ankommt.

Dem entsprechend ist jeder Kunde von Ihnen von dieser Lücke betroffen und gefährdet. Durch ein Ausnutzen dieser Lücke von einem Angreifer können Kunden bestohlen werden und Ihre Bank würde an Prestige einbüßen.

Um die Lücke zu beheben sollte eine kollisionsresistente Hashfunktion genutzt werden. Beispielsweise könnte das Problem durch SHA-3 gelöst werden. Um Änderungen dieser Art zu diskutieren bitten wir um Kontakt zu Ihren Systemadministratoren.

Bis die Lücke behoben ist, sollte das Überweisungssystem, aufgrund der möglicherweise verheerend Schäden, vorübergehend abgeschaltet werden. Eine Lösung kann unserer Einschätzung nach in 14 bis 28 Tagen erarbeitet werden.

Mit freundlichen Grüßen,

Nico Steppeler und Sabri Öztürk, Berater für IT-Sicherheit

5 Zusammenfassung und Fazit

Dieser Versuch verdeutlicht, warum MD5 heutzutage nicht mehr verwendet werden sollte. Digitale Signaturen, die auf dem Hash-Then-Sign-Prinzip basieren, sind unsicher, wenn die verwendete Hash-Funktion nicht ausreichend kollisionsresistent ist. Wir haben uns in die Rolle eines Angreifers versetzt und versucht, eine Banküberweisung so zu manipulieren, dass das Geld am Ende an uns gesendet wird, anstatt an die ursprüngliche Adresse.

Die Überweisung wurde als PostScript-Dateien zusammengestellt und dabei wurde ein MD5-Hashwert der Datei gebildet, um die Integrität sicherzustellen. Wir haben zwei PostScript-Dateien mit demselben MD5-Hashwert erstellt, wofür wir die mangelnde Kollisionsresistenz der MD5-Hashfunktion ausgenutzt haben. Eine enthielt die originale Überweisung und die andere eine manipulierte Version, beide mit gültigen Signaturen. Dies ermöglichte es uns, die erforderlichen Dateien für einen Diebstahl vorzubereiten. Die Signatur an sich mag sicher erscheinen, aber die Verwendung von MD5 erwies sich als anfällig für Angriffe, was das System insgesamt gegen Angriffe auf starke Kollisionsresistenz verwundbar machte.

Solch ein Angriff birgt ein Risiko, insbesondere wenn man bedenkt, dass auch der Betrag geändert werden könnte. Daher sollten robustere Hash-Funktionen eingesetzt werden, die nicht nur primage resistance, sondern auch Kollisionsresistenz bieten, insbesondere gegen starke Kollisionsresistenz.

Der Versuch war äußerst interessant und gut durchgedacht. Zum ersten Mal haben wir eine einzelne Schwachstelle in einem System ausgenutzt, in dem verschiedene kryptografische Werkzeuge eingesetzt wurden, um einen gezielten Angriff zu konzipieren. Bisher hatten wir es immer mit einzelnen Dateien zu tun, die beispielsweise nur signiert wurden. Dabei wurde klar, dass ein System nur so sicher ist wie seine schwächste Stelle.

6 LLM

In diesem Versuch wurden keine Large-Language Models(LLM) verwendet. Die Texte sind alleine die Arbeit der Mitglieder der Gruppe 43 wie zubeginn angegeben.

Literatur

[1] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Bachelor Grundlagenpraktikum ITS: Veranstaltungsseite der Fakultät für Elektrotechnik und Informationstechnik. URL https://informatik.rub.de/studium/lehrveranstaltungen/.

[2] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Aufgabenstellung zum Praktikumsversuch. Bochum, 2023.

[3] Fiona Weber. URL https://de.wikipedia.org/wiki/Kollisionsresistenz.